Тема 1

• namespace – пространство от имена

→ инструмент за избягване на конфликти на имена
→ scope, в който има дефинирани символи

namespace <ns-name> { void f() {...} }

Извикване:

I.	II.	III.
<ns-name>::f();	using <ns-name>::f();	using namespace <ns-name>;
оператор за резолюция	позволява извикването на даденият символ без оп. за рез.	позволява извикването на всички символи от <ns-name> без оп. ::

→ могат да доведат до конфликти

ns A { void f() {...} } ns B { void f() {...} }

using namespace A;
using namespace B; дотук би се компилирало

---

f(); → (не се компилира) конфликт на имена, който може да се избегне:

A::f(); B::f();

→ добра практика е функциите с еднаква семантика да се отделят в един namespace

• енумерация (enum) – тип, ограничен до домейн от стойности, които включват специално дефинирани константи (енумератори)

[enum – имплицитно кастване; unscoped
enum class – експлицитно кастване; scoped]

```
enum Color {
    red, //0 → no default
    blue, //1 (+1)
    orange, //2 (+1)
};
```

```
enum Fruit {
    apple = 10,
    peach, //11
    orange = 2
};
```

конфликт на имена

```
int n = apple;
if (red == apple)
{...}
```

→ валиден синтаксис, който обаче не искаме, затова ползваме <u>enum class</u>

```
enum class Color {
    orange
};
```

```
enum class Fruit {
    orange
};
```

вече няма конфликт на имена ✓

int n = Color::orange; → не се компилира (може само с cast-ване)
→ int n = (int)(Color::orange);

→ размерът на даден enum е равен на големината на целочислен тип, в чийто домейн се побират стойностите на енумераторите

можем ръчно да зададем размера на enum

```
enum class T : char { -> sizeof(T) = sizeof(char)
        a,
        b = 1000 -> 1000 > char_max => не се компилира
};
```

```
enum A { a };   "shadow"
int main() { int a = 1; a++; A::a; }
```

- структури – последователности от полета, които се пазят в паметта в определен ред
  - могат да съдържат променливи от различни типове

```
struct Test {
    int a;
    char ch[10];
    bool b;
};
```

Test obj; -> stack (-> инстанция на Test)

Test* ptr = new Test; -> heap

-> достъп до елементи

obj.a , (*ptr).a , ptr->a

-> подаване във функции

f1(Test obj)	f3(Test& obj)	f5(Test* ptr)
f2(const Test obj)	f4(const Test& obj)	f6(const Test* ptr) – const
↓	↓	↓
по копие	по референция	по указател (nullptr check)

-> Избягваме копията, за да не хабим памет

-> Ако няма да променяме обекта -> винаги const !

→ масиви от инстанции

Test arr[10] → stack

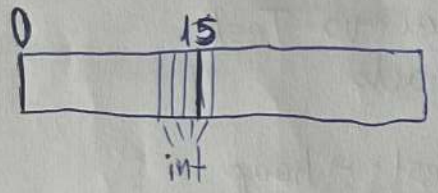
Test* ptr = new Test[n] → heap

→ размер на структура

деф. alignment requirement – разликата між два последователни адреса, на които може да се разположи дадена променлива

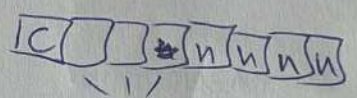
За примитивни типове: sizeof(T) = alignof(T)

→ предотвратява прочитането на дадена променлива от процесора по повече от веднъж



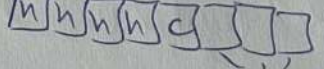
Процесорът чете паметта на блокчета от 16B, т.е., ако int започне на блок 13, ще бъде прочетен два пъти – от 0 до 15 и от 16 до 31.

struct A{ char c; int n; }; →



padding, заради A.R. на int

struct B{ int n; char c }; →



padding

B arr[2]

int, char, int, char – в паметта

Тъй като між елементите на масиви няма място в паметта, всяка инстанция на структура се разполага в паметта по такъв начин, че веднага след нея да може да се разположи друга инстанция. В конкретния пример – след char c се оставя padding, за да може веднага след него да се сложи int n от друга инстанция.

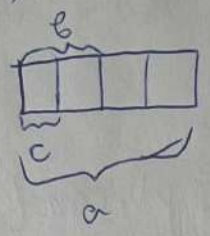
ruct A{ int n; char c; char arr[];}

използва padding-а в края

→ компилира се само ако масивът без размер е единствен и е последен елемента

• union – променливи, които споделят една и съща памет

```
union Test {
  int a;
  short b;
  char c;
};
```



sizeof(Test) = размерът на най-големия тип

→ Endianness – начин на подреждане на байтовете в една процесорна дума

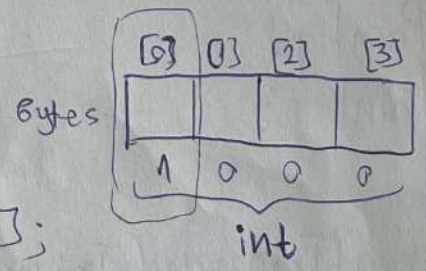
int n = 3;

→ Little endian – [0..11 | 0..0 | 0..0 | 0..000] → приемаме, че използваме него

3 0 0 0

→ Big endian – [0..0 | 0..0 | 0...0 | 0...011]

0 0 0 3

→ проверка:

```
bool isLittleEndian() {
    union EndiannessTest {
        uint32_t = 1;
        unsigned char bytes[4];
    } test;
    return test.bytes[0];
}
```